CAPSTONE PROJECT — APRIL 2026

MedSynth

Comprehensive Architecture & Software Engineering Report

A GAN-powered platform for generating privacy-safe synthetic healthcare data, enabling researchers to train AI models without exposing Protected Health Information.

Project Team

Vansh Sharma	Lakshya Bapna	Sachin Jaiswal	Aditya Mathur	Kushal Sarkar

GitHub Repository	`github.com/adityamathur5836/SD_Capstone`

CONTENTS

01 **Introduction & Problem Statement**

02 **Architectural Design Patterns**

2.1 Model-View-Controller (MVC)

2.2 Singleton Pattern

2.3 Factory Method Pattern

2.4 Strategy Pattern (Polymorphism)

03 **Implementation of SOLID Principles**

04 **System Overview & Technical Diagrams**

Fig. 1 Use Case Diagram

Fig. 2 Entity-Relationship Diagram

Fig. 3 UML Class Diagram

Fig. 4 Sequence Diagram

01 — OVERVIEW

Introduction & Problem Statement

> **MedSynth addresses a critical bottleneck in Healthcare AI:** the tradeoff between Data Privacy and Data Availability. Hospitals possess enormous volumes of patient data (EHRs, imaging) but are barred from sharing them due to strict privacy regulations such as HIPAA and GDPR (Protected Health Information).

MedSynth acts as a **secure intermediary** by manufacturing high-fidelity synthetic healthcare data. Instead of exposing real patient records, the application uses Generative Adversarial Networks — **CTGAN** for tabular data and **DCGAN** for medical imaging — to mathematically learn the statistical representations of diseases.

The system outputs 100% artificial datasets that are statistically identical to the originals, guaranteeing **zero privacy risk** while preventing real patient re-identification. Researchers worldwide can now access medically meaningful data for AI model training without ever touching Protected Health Information.

02 — ARCHITECTURE

Architectural Design Patterns

The distribution of processing logic across MedSynth strictly adheres to proven design patterns, making the codebase highly robust, testable, and scalable.

STRUCTURAL

Model-View-Controller (MVC)

The full-stack application runs on a distributed MVC architecture. The Model (Python ML scripts + database) contains GAN logic. The View (React/Next.js) renders real-time SSE data via visualizations. The Controller (Node.js/Express) accepts HTTP triggers, spawns Model processes, and streams SSE events back.

CREATIONAL

Singleton Pattern

Inside backend/index.js, the MedSynthServer class executes exactly one server instance, securing standard ports, CORS configuration, and unified API routes — preventing double-initializations. The Axios API layer and global Zustand state store on the frontend also use Singleton proxies.

CREATIONAL

Factory Method Pattern

Implemented in script/factory.py. The ModelFactory acts as an object registry — dynamically resolving requests such as create_model("CTGAN") without hard-coding instantiations anywhere in the main loop, completely decoupling object creation from usage.

BEHAVIORAL

Strategy Pattern (Polymorphism)

Defined in script/models.py. The abstract GANModel provides a fixed interface (train() and generate()). Concrete subclasses CTGANModel and DCGANModel act as interchangeable strategies — tabular and image processors run uniquely under a shared parent interface without breaking operations.

03 — ENGINEERING

Implementation of SOLID Principles

The project firmly abides by the SOLID software design philosophy, ensuring every component is clean, maintainable, and future-proof.

S

Single Responsibility Principle (SRP)

Responsibilities are strictly separated. In Node.js, apiRoutes.js solely handles endpoints while jobController.js handles data processing. In Python, MedicalDataProcessor focuses exclusively on PII/PHI scrubbing while AnalysisService handles bias scoring computation independently.

O

Open/Closed Principle (OCP)

The Registry-based ModelFactory is open for extension — simply call .register("NewModel", Class) — but closed for modification. The create_model() function never requires editing fragile if/else blocks when new model types are added to the platform.

L

Liskov Substitution Principle (LSP)

Any pipeline expecting the abstract GANModel can seamlessly receive CTGANModel or DCGANModel without failure. Each subclass perfectly inherits the required baseline contract regardless of internal processing differences between tabular and imaging modalities.

I

Interface Segregation Principle (ISP)

Shared base interfaces enforce only what is appropriate. The Base GAN Interface requests only train() and generate() behaviors, refraining from imposing non-generic, tabular-only algorithms down to image-CNN based modules — keeping interfaces lean and focused.

D

Dependency Inversion Principle (DIP)

High-level execution pipelines rely on abstractions — passing a string constant to ModelFactory — rather than hardcoded import paths to concrete lower-level classes. This safeguards pipelines if concrete implementations change significantly over time.

04 — SYSTEM DIAGRAMS

System Overview & Technical Diagrams

Use Case Diagram

Actor interactions — MedSynth Platform

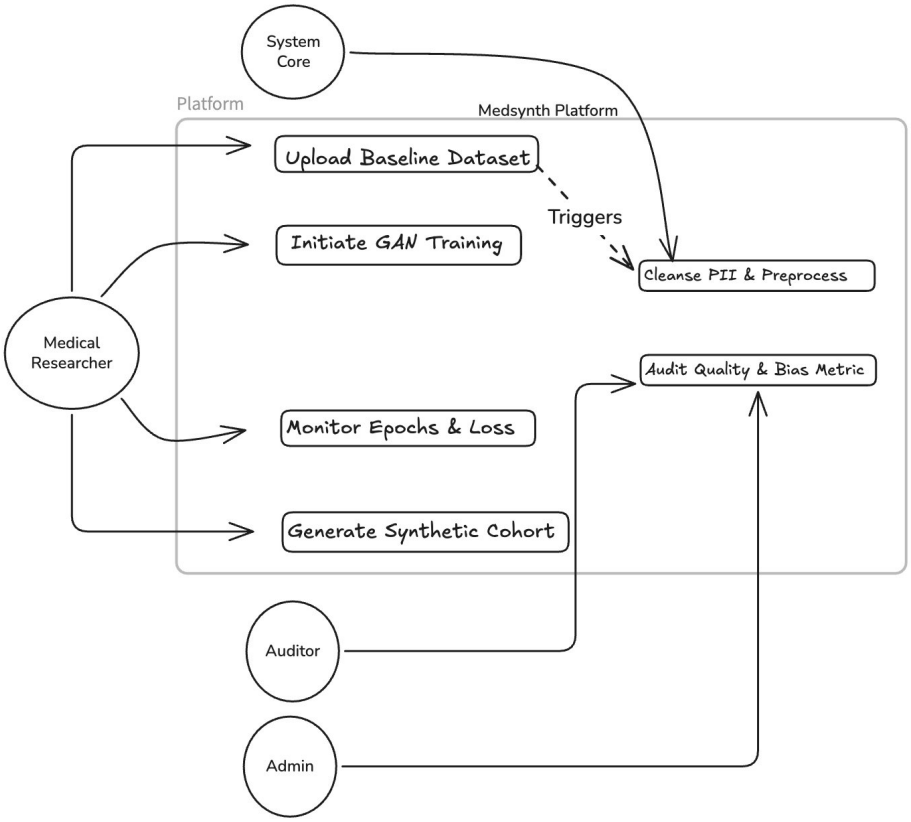

Figure 1 — Illustrates how Medical Researchers, Auditors, and Admins interact with the system. Key flows: upload dataset, initiate GAN training, monitor epochs, generate synthetic cohort, audit quality & bias.

Entity-Relationship (ER) Diagram

PostgreSQL database schema

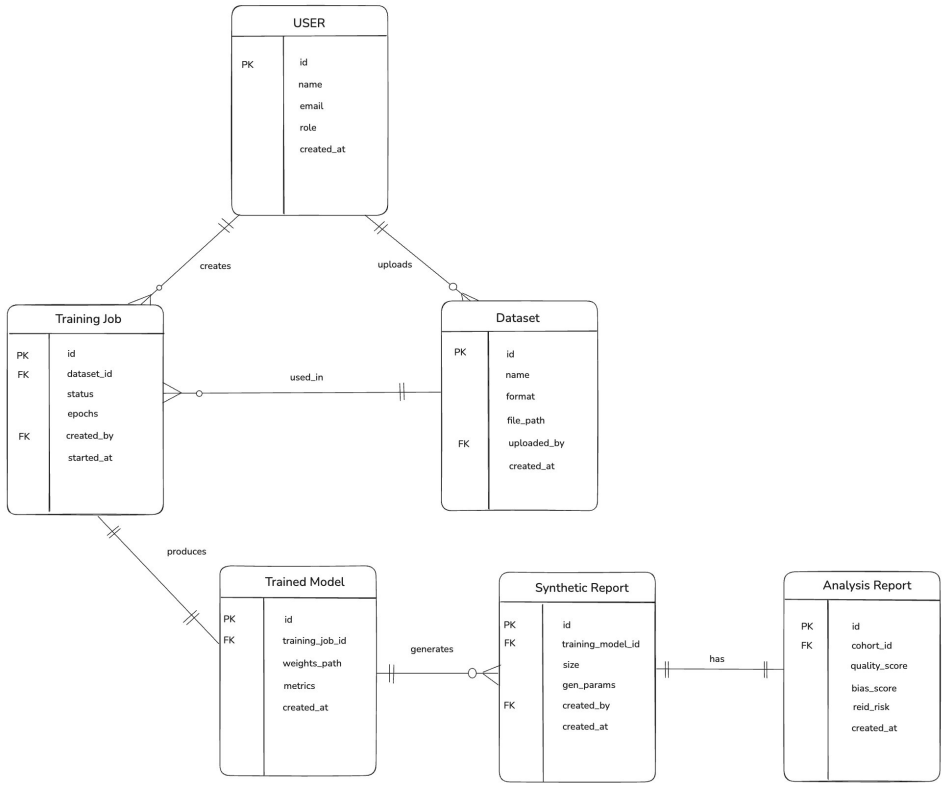

Figure 2 — Users create Training Jobs and upload Datasets. Training Jobs produce Trained Models which generate Synthetic Reports. Each report carries an Analysis Report with quality score, bias score, and re-identification risk.

● UML Class Diagram

Factory & Strategy pattern hierarchy

Figure 3 — Abstract GANModel extended by DCGANModel, CTGANModel, and VAEModel. ModelFactory creates instances via registry. SyntheticDataGenerator orchestrates the pipeline. AnalysisService validates output.

● Sequence Diagram

SSE data flow for training sessions

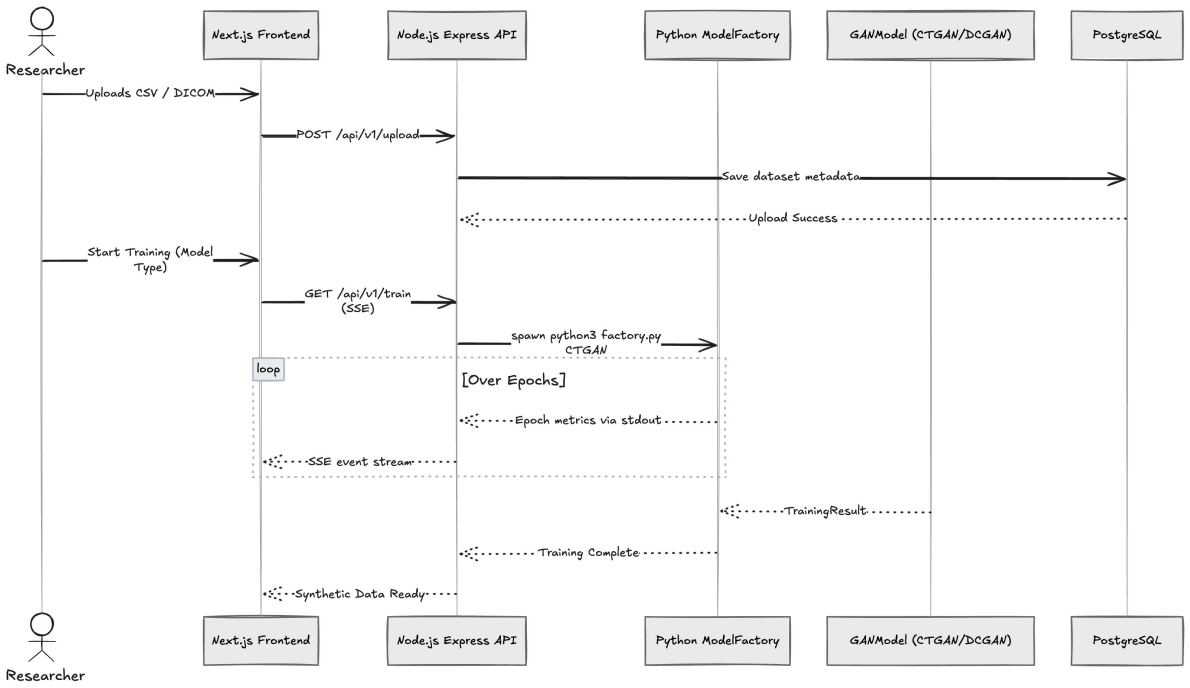

Figure 4 — Full researcher session: CSV/DICOM upload via POST /api/v1/upload, Node.js spawning Python ModelFactory, real-time epoch metrics streamed via SSE, final Synthetic Data Ready notification.